

Online Food Delivery System

Submitted by

Student Name	Student ID
Mir Md Sabit Ishtiaq	253-15-475
Nusrat Jahan Anika	253-15-733
Md. Atik Hasan Anik	253-15-544
Ibthda Thamid Tazim	253-15-353
Md. Shakib All Hasan Mim	252-15-975

LAB PROJECT REPORT

This Report is presented in Partial Fulfillment of the course
CSE124:Data Structure in the Department of Computer Science and Engineering



DAFFODIL INTERNATIONAL UNIVERSITY

Dhaka, Bangladesh

DECLARATION

We hereby declare that this lab project has been done by us under the supervision of **Jamilul Huq Jami (JHJ)**, Lecturer, Department of Computer Science and Engineering, Daffodil International University. We also declare that neither this project nor any part of this project has been submitted elsewhere as a lab project.

Submitted To:

Lecturer,
Department Of Computer Science & Engineering
Daffodil International University

Submitted by

Mir Md. Sabit Ishtiaq ID: 253-15-475 Dept. of CSE, DIU	
Nusrat Jahan Anika ID: 253-15-733 Dept. of CSE, DIU	Md. Atik Hasan Anik ID: 253-15-544 Dept. of CSE, DIU
Md. Shakib All Hasan Mim ID: 252-15-975 Dept. of CSE, DIU	Ibthda Thamid Tazim ID: 253-15-353 Dept. of CSE, DIU

COURSE & PROGRAM OUTCOME

The following courses have course outcomes as following:

Table 1: Course Outcome Statements:

CO's	Statements
CO1	Define and related classes, objects, members of the class, and relationships among them needed for solving specific problems
CO2	Formulate knowledge of programming and c n problem-solving
CO3	Analyze Data structures to present a specific problem

Table 2: Mapping of CO, PO, Blooms, KP, and CEP

CO	PO	Blooms	KP	CEP	CEA
CO1	PO1	C1, C2, P1, P2, P3	K3	EP1	
CO2	PO2	C2, C3, P1, P2, A2	K3, K4	EP2	
CO3	PO3	C3, C6, P1, P2, P3, P4, A1, A2, A3	K5	EP3	

The mapping justification of this table is provided in sections 4.3.1, 4.3.2, and 4.3.3.

Table of Contents:

Content	Page
---------	------

Declaration	02
Course & Program Outcome	03
1. Introduction	
1.1 Introduction	06
1.2 Motivation	06
1.3 Objectives	07
1.4 Feasibility Study	08
1.5 Gap Analysis	09
1.6 Project Outcome	09
2. System Architecture	
2.1 Requirement Analysis & Design Specification	10
2.1.1 Overview	10
2.1.2 System Design	11
2.1.3 UI Design	12
3. Implementation and Results	
3.1 Implementation	14
3.2 Performance Analysis	15
3.3 Results & Discussion	15

4. Engineering Standards And Mapping	
4.1 Impact on Society, Environment, and Sustainability	16
4.1.1. Mapping of Program Outcomes	16
4.1.2 Impact on society and environment	16
4.1.3 Ethical Aspects	17
4.1.4 Sustainability plan	17

4.2 Project Management and Teamwork	17
4.3 Complex Engineering Problem	18
4.3.1 Mapping of program Outcome	18
4.3.2 Complex Problem Solving	19
4.3.3 Engineering activities	19
5. Conclusion	
5.1 Summary	20
5.2 Limitations	22
5.3 Future Work	21
Reference	22

Chapter 1

Introduction

This chapter introduces the **Online Food Delivery System**, including the background of the project, motivation, objectives, feasibility study, gap analysis, and expected project outcomes. The project focuses on applying fundamental data structures to develop a simple and organized food delivery management system.

1.1. Introduction

The **Online Food Delivery System** is a C-based application developed to manage food items, customer accounts, orders, and payment records in an organized way. The system provides a simple platform where food items can be added, customers can place orders, and orders can be processed systematically.

To manage the different operations efficiently, the project uses several data structures. A **Singly Linked List** is used for the dynamic food menu, a **Queue** is used to process customer orders following the **FIFO (First In, First Out)** principle, and a **Stack** is used to maintain payment and transaction history using the **LIFO (Last In, First Out)** principle.

This project demonstrates the practical application of data structures in a real-world food delivery system and helps improve programming and problem-solving skills.

1.2. Motivation

The main motivation behind this project is to understand how fundamental data structures can be applied to a practical real-world application.

In a food delivery system, different types of information require different methods of storage and processing. Food items need to be stored dynamically because new items can be added at any time. Customer orders need to be processed according to their arrival sequence, making a queue suitable for order management. Similarly, recent payment transactions can be accessed efficiently using a stack.

The project was motivated by the following objectives:

- To apply theoretical knowledge of data structures to a practical system.
- To understand the real-world application of linked lists, queues, and stacks.
- To practice dynamic memory allocation using C.
- To develop problem-solving and logical thinking skills.
- To understand how multiple data structures can work together.
- To develop a simple and functional food delivery management system.
- To analyze the efficiency of different data structure operations.

1.3. Objectives

The project was designed with the following objectives:

✓ Functional Objectives

- To manage food items and their prices efficiently.
- To allow customers to view the available food menu.
- To place and manage customer orders.
- To process orders according to the **FIFO** principle.
- To maintain customer account information.
- To maintain payment and transaction records.
- To calculate the total price of customer orders.
- To track total revenue and processed orders.

✓ Technical Objectives

- To implement a **Singly Linked List** for the food menu.
- To implement a **Queue** for customer order processing.
- To implement a **Stack** for payment history.
- To use **dynamic memory allocation** for storing data.
- To implement file handling for customer account storage.
- To analyze the time complexity of major operations.

✓ Security Objective

- To protect customer account information from unauthorized access.
- To maintain customer payment and transaction records securely.
- To provide basic user authentication for customer accounts.
- To improve the overall security and reliability of the system.

1.4. Feasibility Study

✓ Technical Feasibility

The system is developed using **C programming** with standard libraries. It uses Linked List, Queue, Stack, pointers, and dynamic memory allocation, making it technically feasible.

✓ **Economic Feasibility**

The project requires no additional cost. It can be developed and executed using free tools such as **GCC, CodeBlocks, or Visual Studio Code**.

✓ **Operational Feasibility**

The system provides a simple **menu-driven interface** that is easy to understand and operate with minimal instructions.

✓ **Time Feasibility**

The project is moderate in size and can be completed within a reasonable time through proper planning, coding, testing, and documentation.

1.5. Gap Analysis

Although the system provides the basic functions of an online food delivery system, it has some limitations compared to modern food delivery platforms.

✓ **Current Limitations**

- No graphical user interface (GUI)
- Cannot support multiple users at the same time
- Lacks advanced security features like encryption
- No online or real-time transaction system

✓ **Possible Improvements**

- Adding a GUI for a better user experience.
- Integrating a database like MySQL.
- Implementing real-time order and delivery tracking.
- Developing a web or mobile-based version.

1.6. Project Outcome

This project successfully demonstrates how different data structures can be used to develop an **Online Food Delivery System**.

✓ Learning Outcome

- Gained practical knowledge of **Linked List, Queue, and Stack**.
- Improved understanding of **C programming and dynamic memory allocation**.
- Developed problem-solving and logical thinking skills.
- Learned how to analyze the efficiency of data structure operations.

✓ System Outcome

- Provides a functional food menu management system.
- Processes customer orders using a **FIFO Queue**.
- Maintains payment history using a **LIFO Stack**.
- Manages customer accounts and tracks order revenue.

Chapter 2

Proposed Methodology/Architecture

This chapter describes the proposed methodology, system requirements, architecture, and user interface design of the **Online Food Delivery System**. The system uses different data structures according to the requirements of each operation.

2.1 Requirement Analysis & Design Specification

2.1.1 Overview

The proposed **Online Food Delivery System** is designed to provide a simple and organized way to manage food items, customer accounts, orders, and payment records.

- **Technology Stack:** C programming language with `stdio.h`, `stdlib.h`, `string.h`, and `ctype.h`.
- **Core Entities:** Food, Customer, Order, and Payment.
- **Data Management:** Linked List, Queue, and Stack are used for efficient data management.
- **Storage:** Customer account information is stored using `customers.txt`.
- **User Interface:** A simple menu-driven console interface is provided.
- **Memory Management:** Dynamic memory allocation is used to create data nodes.

2.1.2 System Design

The system follows a modular design approach in which different data structures are used according to the requirements of each operation. A **Singly Linked List** is used to manage the food menu dynamically, allowing new food items to be added easily. A **Queue** is used to store and process customer orders following the **FIFO (First In, First Out)** principle. A **Stack** is used to maintain payment and transaction history following the **LIFO (Last In, First Out)** principle. Customer account information is managed using a linked list, while file handling is used to store customer information in `customers.txt`. Dynamic memory allocation is also used to create and manage nodes efficiently.

2.1.3 UI Design

Here are some examples of the UI.

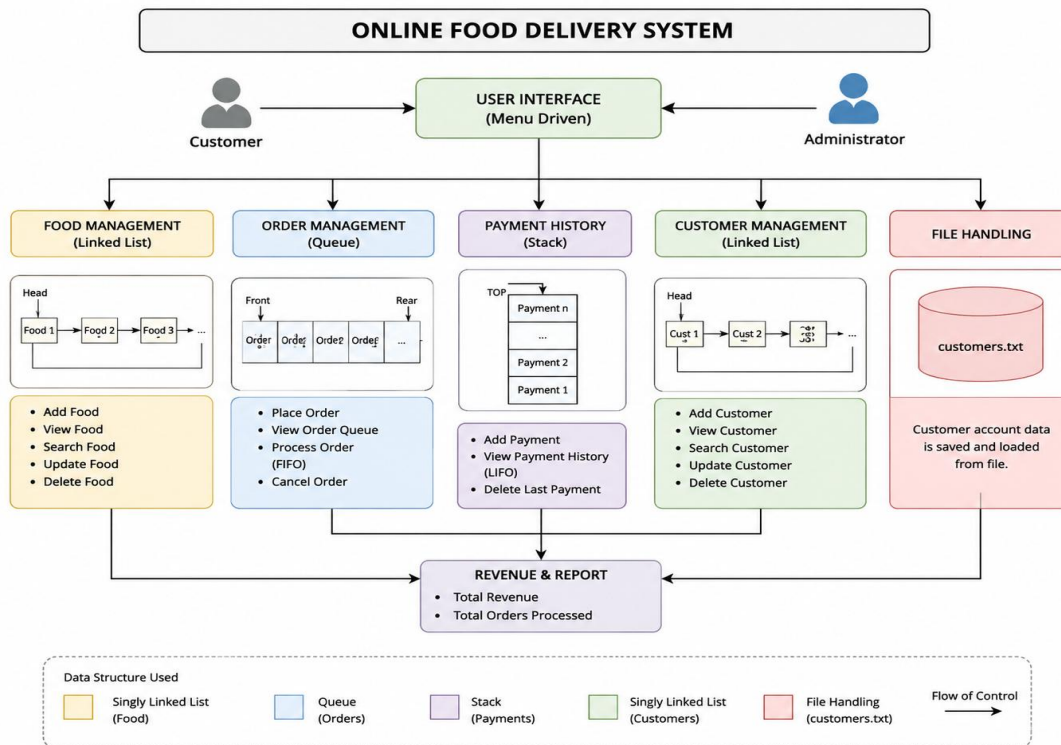


Fig 1: System Diagram

```
===== MAIN MENU =====
1. Admin Panel
2. Customer Panel
0. Exit
Enter choice: |
```

Fig 2: Main Menu

```

===== ADMIN CONTROL PANEL =====
1. Add Food
2. View Menu
3. Search Food
4. Update Food
5. Delete Food
6. View Orders (Queue)
7. Process Next Order (FIFO)
8. Payment History (Stack)
9. Dashboard
0. Logout
Enter choice: |

```

Fig 3: Admin Panel

```

===== CUSTOMER PANEL =====
1. Register
2. Login
3. View Food Menu
4. Search Food
0. Back to Main Menu
Enter choice: |

```

Fig 4: Customer Panel

ID	Food Name	Price
1	Chicken Burger	250.00
2	Beef Kacchi	350.00
3	Veg Fried Rice	180.00
4	Chicken Pizza	550.00
5	Cold Coffee	120.00

Fig 5:View Food Menu

Chapter 3

Implementation and Results

This chapter describes the implementation and performance of the **Online Food Delivery System** developed using C programming and different data structures.

3.1 Implementation

The system is implemented using structured programming and dynamic data structures. A **Singly Linked List** is used for the food menu, a **Queue** is used for customer orders, and a **Stack** is used for payment history.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

/* 1. DATA STRUCTURE DEFINITIONS */

/* ---- Singly Linked List node : Food Menu ---- */
typedef struct FoodNode {
    int id;
    char name[50];
    float price;
    struct FoodNode *next;
} FoodNode;

/* ---- Queue node : Customer Orders (FIFO) ---- */
typedef struct OrderNode {
    int orderId;
    char customerName[50];
    int foodId;
    char foodName[50];
    int quantity;
    float totalPrice;
    struct OrderNode *next;
} OrderNode;

/* ---- Stack node : Payment History (LIFO) ---- */
typedef struct PaymentNode {
    int orderId;
    char customerName[50];
    float amount;
    struct PaymentNode *next;
} PaymentNode;

/* ---- Simple in-RAM customer account list ---- */
```

```

typedef struct CustomerNode {
    char username[30];
    char password[30];
    struct CustomerNode *next;
} CustomerNode;

/* -----
2. GLOBAL POINTERS (heads / front / rear / top)
----- */
FoodNode *menuHead = NULL; /* Linked List head */
FoodNode *menuTail = NULL; /* Linked List tail (for O(1) append) */
OrderNode *queueFront = NULL; /* Queue front */
OrderNode *queueRear = NULL; /* Queue rear */
PaymentNode *stackTop = NULL; /* Stack top */
CustomerNode *custHead = NULL; /* Registered customers */

int nextFoodId = 1;
int nextOrderId = 1;
float totalRevenue = 0.0f;
int totalOrdersProcessed = 0;

const char ADMIN_USER[] = "admin";
const char ADMIN_PASS[] = "admin123";
const char CUSTOMER_FILE[] = "customers.txt"; /* persistent storage for customer accounts */

/* -----
3. UTILITY
----- */
void clearInputBuffer() {
    int c;
    while ((c = getchar()) != '\n' && c != EOF);
}

void pause_screen() {
    printf("\nPress ENTER to continue...");
    clearInputBuffer();
}

/* -----
4. LINKED LIST OPERATIONS -> FOOD MENU
----- */

/* addFood(): O(1) insert at tail using malloc(), so IDs stay in ascending
order (ID 1 always shows first) when the menu is printed.
verbose=1 -> prints a confirmation (used when admin adds an item)
verbose=0 -> silent (used for startup seed data) */
void addFood(const char *name, float price, int verbose) {
    FoodNode *node = (FoodNode *)malloc(sizeof(FoodNode));
    if (node == NULL) {
        printf("Memory allocation failed!\n");
        return;
    }
}

```

```

    }
    node->id = nextFoodId++;
    strcpy(node->name, name);
    node->price = price;
    node->next = NULL;

    if (menuHead == NULL) { /* empty list */
        menuHead = menuTail = node;
    } else {
        menuTail->next = node; /* attach after current tail */
        menuTail = node;
    }
    if (verbose)
        printf("Food item '%s' added successfully with ID %d.\n", name, node->id);
}

/* viewMenu(): traverse the singly linked list */
void viewMenu() {
    if (menuHead == NULL) {
        printf("Menu is currently empty.\n");
        return;
    }
    printf("\n-----\n");
    printf("%-6s %-25s %-10s\n", "ID", "Food Name", "Price");
    printf("-----\n");
    FoodNode *cur = menuHead;
    while (cur != NULL) {
        printf("%-6d %-25s %-10.2f\n", cur->id, cur->name, cur->price);
        cur = cur->next;
    }
    printf("-----\n");
}

/* searchFood(): linear traversal, match by ID or name substring */
FoodNode* searchFoodById(int id) {
    FoodNode *cur = menuHead;
    while (cur != NULL) {
        if (cur->id == id) return cur;
        cur = cur->next;
    }
    return NULL;
}

/* containsIgnoreCase(): manual case-insensitive substring search
(portable — doesn't rely on the non-standard strcasestr) */
int containsIgnoreCase(const char *haystack, const char *needle) {
    if (*needle == '\0') return 1; /* empty search matches everything */
    for (int i = 0; haystack[i] != '\0'; i++) {
        int j = 0;
        while (haystack[i + j] != '\0' && needle[j] != '\0' &&
            tolower((unsigned char)haystack[i + j]) == tolower((unsigned char)needle[j])) {

```

```

        j++;
    }
    if (needle[j] == '\0') return 1; /* matched the whole needle */
}
return 0;
}

void searchFood(const char *keyword) {
    FoodNode *cur = menuHead;
    int found = 0;
    printf("\nSearch results for \"%s\":\n", keyword);
    printf("-----\n");
    while (cur != NULL) {
        if (containsIgnoreCase(cur->name, keyword)) {
            printf("%-6d %-25s %-10.2f\n", cur->id, cur->name, cur->price);
            found = 1;
        }
        cur = cur->next;
    }
    if (!found) printf("No matching food item found.\n");
}

/* updateFood(): find node and update price/name in place */
int updateFood(int id, const char *newName, float newPrice) {
    FoodNode *node = searchFoodById(id);
    if (node == NULL) return 0;
    strcpy(node->name, newName);
    node->price = newPrice;
    return 1;
}

/* deleteFood(): unlink node from the singly linked list and free() it */
int deleteFood(int id) {
    FoodNode *cur = menuHead, *prev = NULL;
    while (cur != NULL && cur->id != id) {
        prev = cur;
        cur = cur->next;
    }
    if (cur == NULL) return 0; /* not found */
    if (prev == NULL) menuHead = cur->next; /* deleting head */
    else prev->next = cur->next;
    if (cur == menuTail) menuTail = prev; /* deleting tail: fix tail pointer */
    free(cur);
    return 1;
}

/* -----
5. QUEUE OPERATIONS (FIFO) -> ORDER DISPATCH
----- */

```

```

/* enqueueOrder(): append at rear in O(1) */

```



```

void enqueueOrder(const char *custName, int foodId, int quantity) {
    FoodNode *food = searchFoodById(foodId);
    if (food == NULL) {
        printf("Invalid Food ID. Order not placed.\n");
        return;
    }
    OrderNode *node = (OrderNode *)malloc(sizeof(OrderNode));
    if (node == NULL) {
        printf("Memory allocation failed!\n");
        return;
    }
    node->orderId = nextOrderId++;
    strcpy(node->customerName, custName);
    node->foodId = foodId;
    strcpy(node->foodName, food->name);
    node->quantity = quantity;
    node->totalPrice = food->price * quantity;
    node->next = NULL;

    if (queueRear == NULL) { /* empty queue */
        queueFront = queueRear = node;
    } else {
        queueRear->next = node;
        queueRear = node;
    }
    printf("Order placed! Order ID: %d | %s x%d | Total: %.2f\n",
        node->orderId, node->foodName, node->quantity, node->totalPrice);
}

/* viewOrders(): traverse the queue from front to rear */
void viewOrders() {
    if (queueFront == NULL) {
        printf("No pending orders in the queue.\n");
        return;
    }
    printf("\n-----\n");
    printf("%-8s %-15s %-20s %-6s %-8s\n", "OrderID", "Customer", "Food", "Qty", "Total");
    printf("-----\n");
    OrderNode *cur = queueFront;
    while (cur != NULL) {
        printf("%-8d %-15s %-20s %-6d %-8.2f\n",
            cur->orderId, cur->customerName, cur->foodName, cur->quantity, cur->totalPrice);
        cur = cur->next;
    }
    printf("-----\n");
}

/* pushPayment(): forward declaration (defined in section 6) */
void pushPayment(int orderId, const char *custName, float amount);

/* processOrder(): dequeue from front (kitchen serves oldest order first),

```

```

    then push its bill onto the payment stack */
int processOrder() {
    if (queueFront == NULL) {
        printf("No orders to process.\n");
        return 0;
    }
    OrderNode *served = queueFront;
    queueFront = queueFront->next;
    if (queueFront == NULL) queueRear = NULL; /* queue became empty */

    printf("Processed Order #%d for %s -> %s x%d (Total: %.2f)\n",
        served->orderId, served->customerName, served->foodName,
        served->quantity, served->totalPrice);

    pushPayment(served->orderId, served->customerName, served->totalPrice);
    totalRevenue += served->totalPrice;
    totalOrdersProcessed++;

    free(served);
    return 1;
}

/* -----
6. STACK OPERATIONS (LIFO) -> PAYMENT / TRANSACTION LEDGER
----- */

/* pushPayment(): O(1) push, attach directly to top */
void pushPayment(int orderId, const char *custName, float amount) {
    PaymentNode *node = (PaymentNode *)malloc(sizeof(PaymentNode));
    if (node == NULL) {
        printf("Memory allocation failed!\n");
        return;
    }
    node->orderId = orderId;
    strcpy(node->customerName, custName);
    node->amount = amount;
    node->next = stackTop;    /* update top */
    stackTop = node;
}

/* viewPaymentHistory(): traverse top -> bottom (most recent first) */
void viewPaymentHistory() {
    if (stackTop == NULL) {
        printf("No completed transactions yet.\n");
        return;
    }
    printf("\n---- Payment History (Most Recent First) ----\n");
    printf("%-8s %-15s %-10s\n", "OrderID", "Customer", "Amount");
    printf("-----\n");
    PaymentNode *cur = stackTop;
    while (cur != NULL) {

```

```

        printf("%-8d %-15s %-10.2f\n", cur->orderId, cur->customerName, cur->amount);
        cur = cur->next;
    }
    printf("-----\n");
}

/* -----
7. CUSTOMER ACCOUNTS (register / login)
----- */

CustomerNode* findCustomer(const char *username) {
    CustomerNode *cur = custHead;
    while (cur != NULL) {
        if (strcmp(cur->username, username) == 0) return cur;
        cur = cur->next;
    }
    return NULL;
}

/* saveCustomerToFile(): append one "username password" line to customers.txt
so the account survives after the program closes */
void saveCustomerToFile(const char *username, const char *password) {
    FILE *fp = fopen(CUSTOMER_FILE, "a"); /* "a" = append, keeps old records */
    if (fp == NULL) {
        printf("Warning: could not save customer data to file.\n");
        return;
    }
    fprintf(fp, "%s %s\n", username, password);
    fclose(fp);
}

/* loadCustomersFromFile(): read customers.txt (if it exists) into the
in-RAM linked list at program startup, so old accounts still work */
void loadCustomersFromFile() {
    FILE *fp = fopen(CUSTOMER_FILE, "r");
    if (fp == NULL) return; /* first run: file doesn't exist yet, that's fine */

    char user[30], pass[30];
    int count = 0;
    while (fscanf(fp, "%29s %29s", user, pass) == 2) {
        CustomerNode *node = (CustomerNode *)malloc(sizeof(CustomerNode));
        strcpy(node->username, user);
        strcpy(node->password, pass);
        node->next = custHead;
        custHead = node;
        count++;
    }
    fclose(fp);
    if (count > 0)
        printf("Loaded %d saved customer account(s) from %s.\n", count, CUSTOMER_FILE);
}

```

```

int registerCustomer(const char *username, const char *password) {
    if (findCustomer(username) != NULL) return 0; /* already exists */
    CustomerNode *node = (CustomerNode *)malloc(sizeof(CustomerNode));
    strcpy(node->username, username);
    strcpy(node->password, password);
    node->next = custHead;
    custHead = node;
    saveCustomerToFile(username, password); /* persist to customers.txt */
    return 1;
}

int loginCustomer(const char *username, const char *password) {
    CustomerNode *c = findCustomer(username);
    if (c == NULL) return 0;
    return strcmp(c->password, password) == 0;
}

/* -----
8. DASHBOARD
----- */
void showDashboard() {
    int menuCount = 0;
    FoodNode *f = menuHead;
    while (f != NULL) { menuCount++; f = f->next; }

    int pendingCount = 0;
    OrderNode *o = queueFront;
    while (o != NULL) { pendingCount++; o = o->next; }

    printf("\n===== ADMIN DASHBOARD =====\n");
    printf(" Food Items in Menu    : %d\n", menuCount);
    printf(" Orders Pending (Queue) : %d\n", pendingCount);
    printf(" Orders Processed       : %d\n", totalOrdersProcessed);
    printf(" Total Revenue          : %.2f\n", totalRevenue);
    printf("===== \n");
}

/* -----
9. SEED DATA (so the menu isn't empty on first run)
----- */
void seedMenu() {
    addFood("Chicken Burger", 250.0f, 0);
    addFood("Beef Kacchi", 350.0f, 0);
    addFood("Veg Fried Rice", 180.0f, 0);
    addFood("Chicken Pizza", 550.0f, 0);
    addFood("Cold Coffee", 120.0f, 0);
}

/* -----
10. ADMIN PANEL

```

```

----- */
void adminPanel() {
    char user[30], pass[30];
    printf("\n----- ADMIN LOGIN ----- \n");
    printf("Username: ");
    scanf("%29s", user);
    printf("Password: ");
    scanf("%29s", pass);

    if (strcmp(user, ADMIN_USER) != 0 || strcmp(pass, ADMIN_PASS) != 0) {
        printf("Invalid admin credentials.\n");
        return;
    }
    printf("Admin login successful!\n");

    int choice;
    do {
        printf("\n===== ADMIN CONTROL PANEL =====\n");
        printf("1. Add Food\n");
        printf("2. View Menu\n");
        printf("3. Search Food\n");
        printf("4. Update Food\n");
        printf("5. Delete Food\n");
        printf("6. View Orders (Queue)\n");
        printf("7. Process Next Order (FIFO)\n");
        printf("8. Payment History (Stack)\n");
        printf("9. Dashboard\n");
        printf("0. Logout\n");
        printf("Enter choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1: {
                char name[50]; float price;
                printf("Food name: ");
                scanf(" %49[^\n]", name);
                printf("Price: ");
                scanf("%f", &price);
                addFood(name, price, 1);
                break;
            }
            case 2:
                viewMenu();
                break;
            case 3: {
                char kw[50];
                printf("Search keyword: ");
                scanf(" %49[^\n]", kw);
                searchFood(kw);
                break;
            }
        }
    }
}

```

```

    case 4: {
        int id; char name[50]; float price;
        printf("Food ID to update: ");
        scanf("%d", &id);
        printf("New name: ");
        scanf(" %49[^\n]", name);
        printf("New price: ");
        scanf("%f", &price);
        if (updateFood(id, name, price))
            printf("Food item updated.\n");
        else
            printf("Food ID not found.\n");
        break;
    }
    case 5: {
        int id;
        printf("Food ID to delete: ");
        scanf("%d", &id);
        if (deleteFood(id))
            printf("Food item deleted.\n");
        else
            printf("Food ID not found.\n");
        break;
    }
    case 6:
        viewOrders();
        break;
    case 7:
        processOrder();
        break;
    case 8:
        viewPaymentHistory();
        break;
    case 9:
        showDashboard();
        break;
    case 0:
        printf("Admin logged out.\n");
        break;
    default:
        printf("Invalid choice.\n");
}
} while (choice != 0);
}

/* -----
11. CUSTOMER PANEL
----- */
/* Shown once a customer has logged in: no more Register/Login options,
   only actions that make sense for an authenticated customer. */
void loggedInCustomerMenu(const char *loggedInUser) {

```

```

int choice;
do {
    printf("\n===== Welcome, %s =====\n", loggedInUser);
    printf("1. View Food Menu\n");
    printf("2. Search Food\n");
    printf("3. Place Order\n");
    printf("4. View Pending Orders\n");
    printf("0. Logout\n");
    printf("Enter choice: ");
    scanf("%d", &choice);

    switch (choice) {
        case 1:
            viewMenu();
            break;
        case 2: {
            char kw[50];
            printf("Search keyword: ");
            scanf(" %49[^\n]", kw);
            searchFood(kw);
            break;
        }
        case 3: {
            int id, qty;
            viewMenu();
            printf("Enter Food ID to order: ");
            scanf("%d", &id);
            printf("Enter quantity: ");
            scanf("%d", &qty);
            if (qty <= 0)
                printf("Quantity must be positive.\n");
            else
                enqueueOrder(loggedInUser, id, qty);
            break;
        }
        case 4:
            viewOrders();
            break;
        case 0:
            printf("Logged out. Returning to Customer Panel.\n");
            break;
        default:
            printf("Invalid choice.\n");
    }
} while (choice != 0);
}

/* Shown before login: Register / Login / browse-without-account only. */
void customerPanel() {
    int choice;

```

```

do {
    printf("\n===== CUSTOMER PANEL =====\n");
    printf("1. Register\n");
    printf("2. Login\n");
    printf("3. View Food Menu\n");
    printf("4. Search Food\n");
    printf("0. Back to Main Menu\n");
    printf("Enter choice: ");
    scanf("%d", &choice);

    switch (choice) {
        case 1: {
            char user[30], pass[30];
            printf("Choose username: ");
            scanf("%29s", user);
            printf("Choose password: ");
            scanf("%29s", pass);
            if (registerCustomer(user, pass))
                printf("Registration successful! You can now log in.\n");
            else
                printf("Username already exists.\n");
            break;
        }
        case 2: {
            char user[30], pass[30];
            printf("Username: ");
            scanf("%29s", user);
            printf("Password: ");
            scanf("%29s", pass);
            if (loginCustomer(user, pass)) {
                printf("Login successful! Welcome, %s.\n", user);
                loggedInCustomerMenu(user); /* switch to the logged-in-only menu */
            } else {
                printf("Invalid username or password.\n");
            }
            break;
        }
        case 3:
            viewMenu();
            break;
        case 4: {
            char kw[50];
            printf("Search keyword: ");
            scanf(" %49[^\n]", kw);
            searchFood(kw);
            break;
        }
        case 0:
            printf("Returning to main menu.\n");
            break;
        default:

```



```

        printf("Invalid choice.\n");
    }
} while (choice != 0);
}

/* -----
12. FREE ALL MEMORY ON EXIT (good practice)
----- */

void freeAll() {
    FoodNode *f = menuHead;
    while (f) { FoodNode *t = f; f = f->next; free(t); }

    OrderNode *o = queueFront;
    while (o) { OrderNode *t = o; o = o->next; free(t); }

    PaymentNode *p = stackTop;
    while (p) { PaymentNode *t = p; p = p->next; free(t); }

    CustomerNode *c = custHead;
    while (c) { CustomerNode *t = c; c = c->next; free(t); }
}

/* -----
13. MAIN
----- */

int main() {
    seedMenu();
    loadCustomersFromFile(); /* restore customer accounts saved from previous runs */

    int choice;
    printf("=====\n");
    printf("    ONLINE FOOD DELIVERY SYSTEM (C Language)\n");
    printf("    Linked List + Queue (FIFO) + Stack (LIFO)\n");
    printf("=====\n");

    do {
        printf("\n===== MAIN MENU =====\n");
        printf("1. Admin Panel\n");
        printf("2. Customer Panel\n");
        printf("0. Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1: adminPanel(); break;
            case 2: customerPanel(); break;
            case 0: printf("Thank you for using the system. Goodbye!\n"); break;
            default: printf("Invalid choice.\n");
        }
    } while (choice != 0);

    freeAll();

```

```
    return 0;  
}
```

3.3 Results and Discussion:

The **Online Food Delivery System** was successfully implemented using C programming and fundamental data structures. The system demonstrates the practical use of a **Singly Linked List, Queue, Stack, and dynamic memory allocation**.

- Food items can be added dynamically to the menu.
- Customer orders can be maintained using a **FIFO Queue**.
- Payment records can be maintained using a **LIFO Stack**.
- Customer account information can be stored using a linked structure.
- The system maintains **total revenue** and **total processed orders**.
- Dynamic memory allocation is used for creating new nodes.

Overall, the implementation demonstrates how different data structures can work together to manage the major components of an Online Food Delivery System.

Chapter 4

Engineering Standards and Mapping

This chapter discusses the engineering aspects of the **Online Food Delivery System**, including its impact on society and environment, ethical considerations, sustainability, teamwork, and mapping with program outcomes.

4.1 Impact on Society, Environment, and Sustainability

4.1.1 Impact on Life

The Online Food Delivery System provides a convenient way to manage food orders digitally. It can reduce manual work and make food ordering and order management more organized and efficient.

4.1.2 Impact on Society & Environment

Societal Impact:

The system provides a basic digital solution for managing food menus, customer orders, and payment records. It can serve as a foundation for developing more advanced food delivery services.

Environmental Impact:

Digital food menus and electronic customer records can reduce the use of paper-based records, supporting a more environmentally friendly approach.

4.1.3 Ethical Aspects

Customer information, including usernames and passwords, should be handled responsibly. The system should protect customer information from unauthorized access and maintain the accuracy and privacy of stored data.

4.1.4 Sustainability Plan

The system can be extended in the future by integrating a database, graphical interface, online payment system, and delivery tracking. Its modular data-structure-based design makes it easier to add new features without changing the complete system.

4.2 Project Management and Teamwork

The development followed an Agile-inspired methodology, ensuring iterative progress and adaptability.

- **Planning:** Identified system requirements and selected Linked List, Queue, and Stack.
- **Development:** Implemented the food menu, customer accounts, order queue, and payment stack.
- **Cost Analysis:** While the current console-based prototype is cost-efficient (focusing primarily on development time), a full-scale rollout would involve investments in secure database servers, advanced encryption layers, and a polished Graphical User Interface (GUI).

4.3 Complex Engineering Problem

4.3.1 Mapping of Program Outcome

The following table justifies how the project satisfies the requirements of specific Program Outcomes.

Table: Justification of Program Outcomes

PO's	Justification
PO1	Engineering Knowledge: Applied advanced mathematical logic and computing principles to manage financial data structures and transaction algorithms effectively
PO2	Problem Analysis: Identified and analyzed the complexities of concurrent banking transactions, ensuring data consistency and systemic reliability.
PO3	Design/Development of Solutions: Designed a comprehensive system that meets specific needs for security and efficiency while considering public health, safety, and environmental constraints.

4.3.2 Complex Problem Solving

The project addresses several "Complex Problem" attributes, ranging from depth of knowledge to stakeholder involvement.

Table: Mapping with Complex Problem Solving

EP1 Depth of Knowledge	EP2 Range of Conflicting Requirements	EP3 Depth of Analysis	EP4 Familiarity with Issues	EP5 Extent of Applicable Codes	EP6 Extent of Stakeholder Involvement	EP7 Interdependence
✓	✓	✓	✓			✓

4.3.3 Engineering Activities

The following mapping highlights the nature of the engineering activities involved in this project.

Table: Mapping with Complex Engineering Activities

EA1 Range of Resources	EA2 Level of Interaction	EA3 Innovation	EA4 Consequences for Society	EA5 Familiarity

Chapter 5

Conclusion

This chapter summarizes the **Online Food Delivery System**, its limitations, and possible future improvements. The project demonstrates the practical use of fundamental data structures in developing a real-world application.

5.1 Summary

The **Online Food Delivery System** is a C-based application developed to manage food items, customer accounts, orders, and payment records. The system uses a **Singly Linked List** for the food menu, a **Queue** for customer orders, and a **Stack** for payment history.

The project helped us understand the practical application of data structures, dynamic memory allocation, structures, pointers, and file handling. Overall, the system provides a simple and organized approach to managing the basic operations of an online food delivery service.

5.2 Limitation

The Bank Account Management System has some limitations:

- The system uses a **console-based interface** instead of a graphical interface.
- It does not include a real-time delivery tracking system.
- No real online payment gateway is integrated.
- Customer data is stored using a local file instead of a database.
- Advanced security features such as encryption and multi-factor authentication are not implemented.

5.3 Future Work

The system can be improved in the future by adding:

- **Database Integration:** MySQL or PostgreSQL can be used for better data management.
- **Graphical/Web Interface:** A GUI or web-based system can make the application more user-friendly.
- **Online Payment:** Real payment gateway integration can be added.
- **Delivery Tracking:** Real-time order and delivery tracking can be implemented.
- **Enhanced Security:** Password encryption, OTP verification, and other security features can be introduced.
- **Mobile Application:** A mobile version can make the system more accessible to customers.

References

<https://www.geeksforgeeks.org/>

https://www.w3schools.com/c/c_projects.php